New Relic APM — Day 5 Study Guide: Advanced NRQL, Integrations and Capstone

Oct 29, 2026 · @Vivek Arora

Day 5 at a glance

Days 1–4 built the parts: APM, errors and alerts, runtime and database, dashboards, custom instrumentation, browser and synthetics. Day 5 puts them to work the way a platform team does: asking hard questions of the data with NRQL, connecting New Relic to the tools the organisation already uses, and running a capstone that turns a monitored application into service levels, an error budget and a performance report you can present.

Agenda

Time	Session	Lab
09:30 – 09:45	Day 4 recap; check the lab system	—
09:45 – 10:30	**Module 13** — Advanced NRQL	—
10:30 – 11:30	Lab 5.1 — Answer production questions with advanced NRQL	Lab 5.1 (60 min)
11:30 – 11:45	Break	—
11:45 – 12:45	**Module 14** — Integrations and AIOps	—
12:45 – 13:45	Lunch	—
13:45 – 14:45	Lab 5.2 — Slack and incident tools, CI/CD deployment tracking, Kubernetes	Lab 5.2 (60 min)
14:45 – 15:00	Break	—
15:00 – 15:30	**Module 15** — Capstone use case and strategy	—
15:30 – 16:45	Lab 5.3 — Capstone: service levels, error budget, reporting and recommendations	Lab 5.3 (75 min)
16:45 – 17:30	Capstone presentations, course review and close	—

Learning objectives

By the end of Day 5 you can:

1. Combine advanced filters, nested aggregations and time comparisons to answer questions no built-in page answers.
2. Use funnels, percentiles and histograms to describe user journeys and latency distributions honestly.
3. Transform data in a query with `filter()`, facet functions and `rate()`, and know when each one is the right tool.
4. Route issues to Slack, PagerDuty and ServiceNow through destinations and workflows, without duplicate or noisy notifications.
5. Record deployments from a CI/CD pipeline, and connect AWS, Azure and Kubernetes telemetry to the services that run on them.
6. Explain what incident intelligence, correlation and baselines do, and where they need human tuning.
7. Set up New Relic end to end for a microservice application, define service levels and an error budget, and present performance findings as evidence-backed recommendations.

How to use this guide with the lab guide

This study guide explains the concepts; every command, click and expected screen is in the separate **New Relic APM — Day 5 Lab Guide**. Each module ends with a *"Do Lab 5.x"* pointer. The NRQL Reference Guide from earlier in the course is the companion for Module 13.

What the lab system gains today

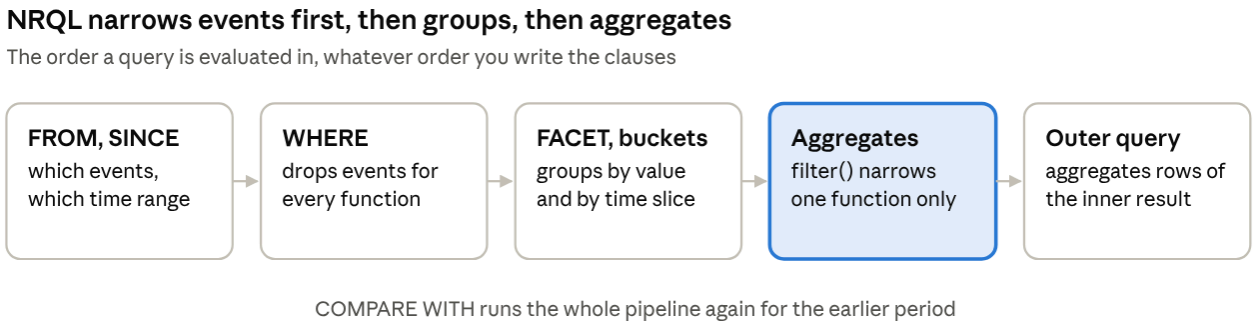
Addition	Why	Lab
Slack destination and workflow (free Slack workspace)	Notifications where the team works; PagerDuty and ServiceNow follow the same pattern if your organisation has them	5.2
A deployment script that records each PetClinic release with NerdGraph	Deployment markers from the pipeline, not by hand	5.2
A local Kubernetes cluster (kind) with the New Relic Helm bundle	Cluster, node, pod and container data next to the services	5.2
Service levels for easyTravel and PetClinic, with burn-rate alerts	The error budget at the centre of the capstone	5.3

The kind cluster needs about 2 GB of memory and 4 GB of disk; stop the headless browsers from Day 4 if the VM is short of memory.

Module 13 — Advanced NRQL usage

The built-in pages answer the questions New Relic expected you to ask. Production asks others: *which customers were affected, in which step did they drop out, is today worse than a normal Tuesday, how many requests were slow — not on average, but really?* This module builds on the NRQL basics from Days 1–3 and the NRQL Reference Guide.

One idea makes advanced NRQL predictable: the order in which a query is evaluated.



NRQL evaluation order

A `WHERE` condition removes events before anything is counted; a `filter()` inside one function narrows only that function. That is why `filter()` can put a numerator and a denominator side by side in one query, and `WHERE` cannot.

13.1 Advanced filtering, aggregations and time comparisons

Filtering. Beyond `=` and `AND`:

Need	NRQL
Several values	`WHERE appName IN ('easyTravel-frontend', 'easyTravel-backend')`
Pattern	`WHERE name LIKE '%OwnerSearch%'` · `WHERE request.uri RLIKE '/owners/[0-9]+'` (regular expression)
Missing or present	`WHERE error.expected IS NULL` · `WHERE nr.syntheticsMonitorId IS NOT NULL`
Numeric ranges	`WHERE duration > 1 AND duration <= 3`

Need	NRQL
Values from another query	`WHERE trace.id IN (FROM Span SELECT uniques(trace.id) WHERE db.statement LIKE '%owners%' LIMIT MAX)`

Aggregations. Count and average are where you start, not where you stop: `uniqueCount()` for distinct users or hosts, `latest()` and `earliest()` for state and first-seen, `percentage()` for shares, `stddev()` for variability, `apdex()` for satisfaction, `cdfPercentage(duration, 0.5, 1)` for "what share was faster than 0.5 s and 1 s".

Nested aggregation — aggregate the result of an aggregation. The inner query produces rows (one per facet and time slice); the outer query works on those rows:

```
-- The busiest minute of the day, per service, in requests per minute
SELECT max(rpm) FROM (
  SELECT count(*) AS rpm FROM Transaction
  WHERE appName IN ('easyTravel-frontend', 'petclinic') FACET appName
TIMESERIES 1 minute
) FACET appName SINCE 1 day ago

-- Average error rate across hosts, not across requests
SELECT average(errPct) FROM (
  SELECT percentage(count(*), WHERE error IS true) AS errPct FROM Transaction
FACET host
) SINCE 1 hour ago
```

Time comparisons.

- `COMPARE WITH 1 week ago` runs the same query for the earlier period and shows both; on a time series, the earlier one is a dashed line. Compare with the same weekday to avoid comparing a Monday with a Sunday.
- `SINCE '2026-10-28 09:00:00' UNTIL '2026-10-28 11:00:00' WITH TIMEZONE 'Asia/Kolkata'` pins an incident window exactly, in the team's time zone.
- `TIMESERIES 5 minutes SLIDE BY 1 minute` smooths a noisy series with overlapping windows — the same idea as an alert condition's sliding window.
- `TIMESERIES MAX` uses the finest buckets the time range allows.

13.2 Funnel analysis, percentiles and histograms

Funnels count how many distinct *somethings* (sessions, users, orders) reached each step. Each step is a `WHERE` condition; a session counts at step *n* only if it also matched every step before it.

```
SELECT funnel(session,
  WHERE pageUrl LIKE '%:8090/' AS 'Home',
  WHERE pageUrl LIKE '%/owners/find' AS 'Find owners',
  WHERE name LIKE '%OwnerSearch/many' AS 'Results',
  WHERE pageUrl RLIKE '.*/owners/[0-9]+' AS 'Owner details')
FROM PageView WHERE appName = 'petclinic' SINCE 1 day ago
```

Read the drop between steps, not the totals: a step that loses half the sessions is the place to look.

Percentiles. An average of 300 ms can hide 5 % of requests taking 4 s. Report the median for the typical experience and p95 or p99 for the tail — and compare percentiles of the same thing over time, never an average with a percentile.

```
SELECT percentile(duration, 50, 90, 95, 99) FROM Transaction
WHERE appName = 'easyTravel-frontend' TIMESERIES SINCE 3 hours ago
```

Histograms show the whole distribution. Two peaks (a fast cached path and a slow uncached one) look like one average and one percentile, but the histogram shows both:

```
-- 20 buckets from 0 to 2 seconds
SELECT histogram(duration, 2, 20) FROM Transaction
WHERE appName = 'easyTravel-backend' SINCE 1 hour ago
```

13.3 Data transformation with filter(), facet functions and rate()

Function	What it does	Example
`filter(fn, WHERE …)`	Applies one aggregate to a subset; several in one query give ratios	`filter(count(*), WHERE error IS true) / count(*) * 100`
`FACET CASES(WHERE … AS 'x', …)`	Groups by conditions you define instead of raw values	Fast / OK / slow bands; internal vs external users
`FACET buckets(attr, ceiling, n)`	Groups a numeric attribute into ranges	`FACET buckets(duration, 3, 6)`
`capture(attr, r'regex')`	Extracts part of a string with a named group	`FACET capture(request.uri, r'/(?P<section>[^/]+)/.*')`
`rate(fn, interval)`	Expresses a count as a rate per interval	`rate(count(*), 1 minute)` = requests per minute

Function	What it does	Example
`derivative(attr, interval)`	Rate of change of a gauge	Growth of heap or disk per hour

```
-- Error rate and slow rate side by side, per service
SELECT filter(count(*), WHERE error IS true) / count(*) * 100 AS 'error %',
       filter(count(*), WHERE duration > 1) / count(*) * 100 AS 'slow %',
       rate(count(*), 1 minute) AS 'rpm'
FROM Transaction WHERE appName IN ('easyTravel-frontend', 'easyTravel-
backend', 'petclinic')
FACET appName SINCE 1 hour ago

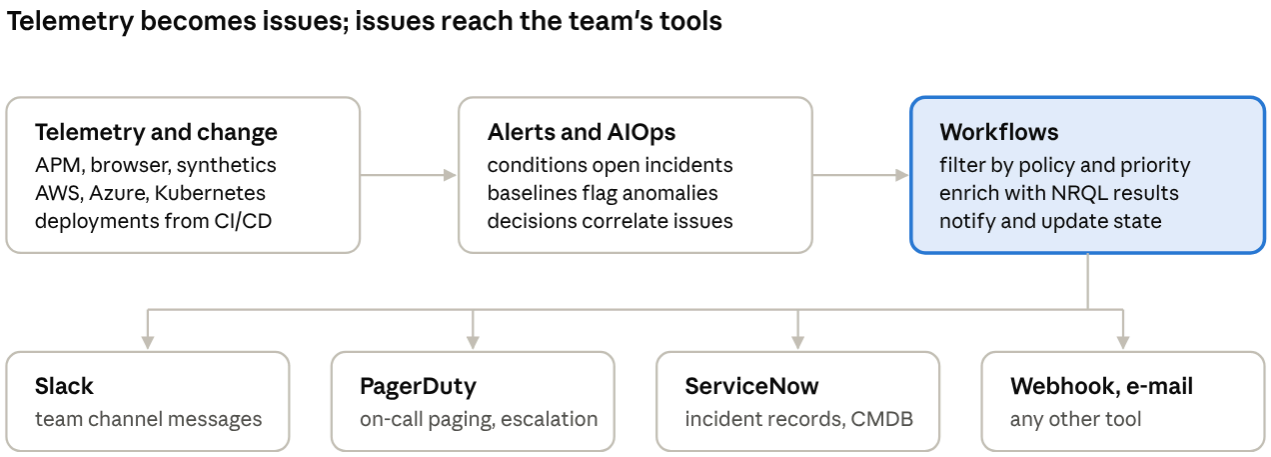
-- Requests grouped by experience band
SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-frontend'
FACET CASES(WHERE duration <= 0.5 AS 'fast', WHERE duration <= 2 AS
'tolerable', WHERE duration > 2 AS 'slow')
SINCE 1 day ago
```

`FACET CASES` assigns each event to the *first* case it matches, so order the cases from narrowest to widest.

> **Do Lab 5.1 — Answer production questions with advanced NRQL** in the Day 5 Lab Guide (60 min): use the easyTravel and PetClinic data from Days 1–4 to answer seven questions an incident commander or product owner would ask — with nested aggregation, time comparisons, a PetClinic user funnel, percentiles and histograms, and `filter()`, `CASES`, `buckets()`, `capture()` and `rate()`.

Module 14 — Integrations and AIOps

New Relic is one tool among many. Engineers talk in Slack, on-call runs in PagerDuty, incidents and changes are recorded in ServiceNow, releases come from a CI/CD pipeline, and the applications run on AWS, Azure or Kubernetes. Integrations put New Relic's data and decisions into those tools, and bring their context back.



integrations around the alerting pipeline

14.1 Slack, ServiceNow and PagerDuty

All three follow the pattern from Day 2: a **destination** holds the connection (credentials, instance, workspace), a **workflow** decides which issues go there and what they carry. One issue — not one incident or one condition — is the unit that is sent, so correlation (Module 14.4) directly reduces notifications.

Tool	Destination needs	Typical use	Watch out for
Slack	The New Relic app installed in the workspace (OAuth), then a channel chosen in the workflow	Team awareness: every issue for the service in its channel, with a link and the enrichment	A channel that receives everything is muted within a week; one channel per team or service
PagerDuty	An API key with account scope, or a service's Events API v2 integration key	Paging the on-call engineer for critical issues; PagerDuty owns escalation and schedules	Map New Relic priority to PagerDuty urgency; do not page from Slack-level issues
ServiceNow	The instance URL and an integration user (or OAuth); the field mapping for the incident	An ITSM incident record for each P1/P2, with assignment group and configuration item	Agree the field mapping and the state sync with the ServiceNow team before go-live

Design the routing once, per severity, and write it down:

- **Critical / P1** → PagerDuty (page) + ServiceNow (record) + the service's Slack channel (awareness).

- **High / warning** → Slack only, during working hours.
- Every route carries the runbook link, the issue link and an NRQL enrichment such as the top errors, so the first message already points somewhere.
- Test each workflow from the workflow page before you rely on it, and again after any change in the other tool.

14.2 CI/CD integrations for deployment tracking

Day 2 showed why deployment markers matter: the first question in any incident is *what changed?* Recording markers by hand does not survive a busy week, so the pipeline records them.

Method	When
NerdGraph mutation (`changeTrackingCreateDeployment`, or the newer `changeTrackingCreateEvent`)	Any pipeline that can run `curl`
`newrelic` CLI (`newrelic changeTracking create`)	Build agents with the CLI installed (Day 2)
GitHub Action `newrelic/deployment-marker-action`	GitHub Actions workflows
Plugins and scripts for Jenkins, GitLab CI, Azure DevOps, Argo CD	Use the same mutation underneath

```
# GitHub Actions: record the deployment after the deploy job succeeds
- name: New Relic deployment marker
  uses: newrelic/deployment-marker-action@v2   # pin to a release tag or
commit SHA
  with:
    apiKey: ${{ secrets.NEW_RELIC_API_KEY }}
    guid: ${{ secrets.NEW_RELIC_ENTITY_GUID }}
    version: ${{ github.ref_name }}
    commit: ${{ github.sha }}
    user: ${{ github.actor }}
```

A good marker carries the **version** (the same value the agent reports as `tags.service.version`), the **commit**, a short **changelog**, the **user** or pipeline and a **deep link** to the build. The API key is a pipeline secret, never a value in the workflow file.

With markers in place, the change tracking view and the APM summary show before/after response time, throughput and errors for every release, and alert issues show the deployments that happened just before them.

14.3 Infrastructure integrations: AWS, Azure and Kubernetes

Platform	How the data arrives	What you get
AWS	CloudWatch **metric streams** through Kinesis Data Firehose (recommended), or API polling for services and inventory; an IAM role with read access	Metrics and inventory for EC2, RDS, Lambda, ELB, SQS and more, as entities related to your services
Azure	The Azure Monitor integration polling with a service principal, or the Azure Native New Relic Service in the Azure portal	Metrics and inventory for VMs, App Service, SQL, Functions, Service Bus and more
Kubernetes	The `nri-bundle` Helm chart: infrastructure agent DaemonSet, kube-state-metrics, Kubernetes events, Prometheus metrics, logs	Cluster explorer: cluster, node, namespace, deployment, pod and container health; events; workloads

The value is in the relationships, not the extra charts. When an APM service runs on a pod, on a node, in a cluster, in an AWS account, New Relic relates those entities: the service map and the *related entities* panel take you from a slow transaction to the saturated node or the throttled RDS instance underneath it, in two clicks.

For Kubernetes, APM agents can be added without rebuilding images by the Kubernetes APM auto-attach operator, which injects the language agent into annotated pods — useful when many teams deploy many services.

14.4 AI, incident intelligence and baselining

Capability	What it does	Where humans stay in charge
Baselines (anomaly thresholds)	Learn each signal's normal pattern, with seasonality, and alert on departures (Day 2)	Choose the direction, the sensitivity and which signals deserve one
Correlation (decisions)	Merge related incidents into one issue by time, topology, similar attributes or your own rules	Review merged issues; disable decisions that join unrelated problems

Capability	What it does	Where humans stay in charge
Issue analysis	The issue page shows related deployments, error groups, anomalous signals and affected entities, and the AI assistant can summarise and suggest causes	Verify each suggestion against the data before acting on it
Proactive detection	Highlights unusual changes in golden signals across services before any condition is set	Decide which findings become real alert conditions
Natural-language queries	The AI assistant turns a question into NRQL	Read the generated query: the filter and the time range decide what the answer means

Measure whether AIOps helps: count notifications per real problem before and after enabling correlation (`NrAiIssue` and `NrAiIncident`, Day 2), and review merged issues weekly for the first month.

> **Do Lab 5.2 — Integrations** in the Day 5 Lab Guide (60 min): route PetClinic issues to a Slack channel (PagerDuty and ServiceNow optional), record releases from a deployment script with NerdGraph, install the Kubernetes integration on a local kind cluster, and look at how correlation and issue analysis present a combined incident.

Module 15 — Capstone use case and strategy

The capstone treats the lab system as a real customer engagement: a set of microservices — easyTravel's frontend and backend with MongoDB, and Spring PetClinic with PostgreSQL, on one Linux host with a Kubernetes cluster beside it — that the business wants monitored, measured against targets, and improved. The deliverable is not more charts; it is a monitoring setup a team can run, an error budget the business agrees to, and a short list of evidence-backed improvements.

15.1 End-to-end setup of New Relic on a microservice application

Work through the setup in layers. Each layer should be finished — consistent, named, tagged — before the next one is built on it.

Layer	Done when	Course day
1. Instrument services	Every service has an APM agent with a clear `app_name`, distributed tracing on, logs in context, and `service.version` set by the deployment	1, 2
2. Instrument the edges	Browser monitoring on user-facing apps; synthetic checks for availability and the critical journeys	4
3. Instrument what they run on	Hosts, containers and Kubernetes; databases (PostgreSQL integration); cloud accounts	3, 5
4. Add business context	Custom attributes and events for the critical transactions; transaction names that separate behaviours	4
5. Organise	Tags on every entity (`team`, `env`, `tier`), workloads per product, naming conventions written down	2
6. Measure	Service levels with targets for each critical journey	5
7. Alert and route	Burn-rate and golden-signal conditions per service, synthetics failures, routed by severity to Slack, PagerDuty and ServiceNow	2, 5
8. Show	On-call, SLO, business and executive dashboards, permissions set, JSON in Git	3
9. Govern	Keys in secret stores, least-privilege users, data ingest reviewed by source, an owner for every dashboard and policy	all

15.2 Custom dashboards and the alerting policy

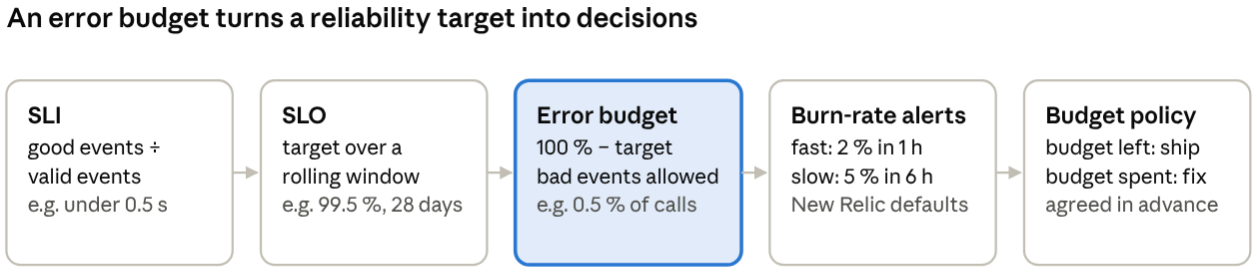
The capstone dashboard extends the Day 3 command center into four pages, each for a different reader:

Page	Reader	Content
On-call	Whoever holds the pager	Golden signals per service, thresholds matching alerts, linked facets, runbook links (Day 3)

Page	Reader	Content
Service levels	Service owners	SLI attainment, error budget left, burn rate, SLO compliance per journey
Business	Product owners	Custom events (`OwnerSearch` outcomes), funnel conversion, page views, Core Web Vitals
Executive	Management	28-day availability against target, incidents and their customer impact, trend versus last month

The alerting policy follows one rule: **alert on symptoms users feel, investigate causes on dashboards.** For each critical journey: a fast-burn and a slow-burn service level alert, the synthetic journey failure condition, and the golden-signal conditions from Day 2. Infrastructure conditions (disk, memory, pod restarts) go to the owning team's Slack channel, not to the pager, unless they always mean user impact.

15.3 Designing error budgets and the performance reporting strategy



from SLI to budget policy

Service level indicators (SLIs) count events: a *valid* event is a request that counts (web transactions of the service), a *good* event is one the user was happy with (no error; faster than the latency threshold). New Relic's service level management defines them as two NRQL conditions and suggests defaults for APM services: *success* (bad events from `TransactionError` where `error.expected != true`) and *latency* (good events with `duration` below a threshold, often started at the recent p95).

The service level objective (SLO) is the target share of good events over a rolling window, for example 99.5 % over 28 days. The **error budget** is what is left: 0.5 % of valid requests may be bad. At 1,000,000 requests in 28 days, that is 5,000 bad requests.

Burn rate says how fast the budget is being used: a burn rate of 1 uses exactly the budget in the window. New Relic's default alerts fire when **2 % of the budget is used within 1 hour** (fast burn: at that pace the budget is gone in about 50 hours) or **5 % within 6 hours**

(slow burn: gone in about 5 days). For a 28-day window (672 hours), 2 % in one hour is a burn rate of 0.02 × 672 = 13.44; with a 0.5 % budget, the fast-burn condition fires when more than 13.44 × 0.5 % ≈ 6.7 % of an hour's requests are bad.

The budget policy is agreed with the business before it is needed:

Budget left	What the team does
More than 50 %	Ship normally; spend some budget on experiments
25 – 50 %	Review the biggest consumers of budget; extra care with risky changes
Under 25 %	Freeze non-essential releases; reliability work first
Spent	Reliability work only until the SLI recovers; post-incident reviews for the main consumers

Choose targets from data, not wishes: look at the last 28 days of the SLI first. A target the service has never met produces permanent alerts; a target it always beats by a mile protects nothing.

Performance reporting strategy — one source of numbers, three audiences:

Report	Cadence	Contents
Operations review	Weekly	SLO status and budget burn, incidents and time to acknowledge and resolve, top regressions, noisy alerts to tune
Service review	Monthly	SLI trends, budget consumption by cause (release, dependency, capacity), capacity outlook, actions and owners
Executive summary	Quarterly	Availability and latency against targets, customer impact, the three improvements delivered, the investment asked for

Every number in a report comes from a saved NRQL query or dashboard widget, so anyone can check it and the next report uses the same definition.

15.4 Presenting performance improvement suggestions

A recommendation is persuasive when the audience can follow the evidence from the user's experience to the fix. Use the same structure for each one:

Part	Question it answers	Example from this course
Observation	What did we see, where?	Owner search p95 430 ms, 86 % of PetClinic database time in one statement (Lab 3.2)
Impact	Who is affected, how much?	Every search; the slowest page in the critical journey; most of the latency budget
Cause	Why, with evidence?	Sequential scan of 2 million rows; the index is unusable for `LIKE 'prefix%'` under the collation
Recommendation	What should change?	A `text_pattern_ops` index, built concurrently
Effort and risk	What does it cost, what could go wrong?	One migration; low risk; reversible
Verification	How will we know it worked?	Owner search database time below 10 ms; latency SLI above target for 7 days

Rank the recommendations by impact against effort, lead with the two or three that matter, and show one before/after chart per recommendation. Close with what you need: a decision, a priority, an owner.

> **Do Lab 5.3 — Capstone** in the Day 5 Lab Guide (75 min, plus presentations): check the end-to-end setup against the layers above, define latency and success service levels for the critical journeys with burn-rate alerts, build the service-level and business pages, write a one-page performance report, and present three improvement recommendations with evidence.

Day 5 cheat sheets

Advanced NRQL patterns — Module 13

```
-- Ratio of two subsets in one query
SELECT filter(count(*), WHERE error IS true) / count(*) * 100 AS 'error %'
FROM Transaction FACET appName

-- Nested aggregation: peak minute per service
SELECT max(rpm) FROM (SELECT count(*) AS rpm FROM Transaction FACET appName
```

```
TIMESERIES 1 minute) FACET appName SINCE 1 day ago

-- Same weekday last week, smoothed
SELECT percentile(duration, 95) FROM Transaction WHERE appName = 'petclinic'
TIMESERIES 10 minutes SLIDE BY 2 minutes SINCE 6 hours ago COMPARE WITH 1
week ago

-- An exact incident window, in the team's time zone
SELECT count(*) FROM TransactionError FACET error.class
SINCE '2026-10-28 16:28:00' UNTIL '2026-10-28 16:48:00' WITH TIMEZONE
'Asia/Kolkata'

-- Funnel of sessions through the critical journey
SELECT funnel(session, WHERE pageUrl LIKE '%/owners/find' AS 'Find', WHERE
name LIKE '%OwnerSearch/many' AS 'Results',
              WHERE pageUrl RLIKE '.*/owners/[0-9]+' AS 'Details') FROM
PageView WHERE appName = 'petclinic' SINCE 1 day ago

-- Distribution and tail
SELECT histogram(duration, 2, 20), percentile(duration, 50, 95, 99) FROM
Transaction WHERE appName = 'easyTravel-backend'

-- Share of requests faster than two thresholds
SELECT cdfPercentage(duration, 0.5, 1) FROM Transaction WHERE appName =
'easyTravel-frontend' SINCE 1 day ago

-- Grouping by your own bands, ranges and extracted parts
SELECT count(*) FROM Transaction FACET CASES(WHERE duration <= 0.5 AS 'fast',
WHERE duration <= 2 AS 'tolerable', WHERE duration > 2 AS 'slow')
SELECT count(*) FROM Transaction FACET buckets(duration, 3, 6)
SELECT count(*) FROM Transaction FACET capture(request.uri, r'/(?P<section>
[^/]+).*')

-- Rates
SELECT rate(count(*), 1 minute) AS 'rpm', rate(filter(count(*), WHERE error
IS true), 1 minute) AS 'errors/min' FROM Transaction FACET appName

-- Values from another event type
SELECT count(*) FROM Transaction WHERE traceId IN (FROM Span SELECT
uniques(trace.id) WHERE name LIKE '%owners%select' AND duration.ms > 100
LIMIT MAX)
```

NerdGraph snippets — Modules 14 and 15

Send each with `curl -s https://api.newrelic.com/graphql -H 'Content-Type: application/json' -H "API-Key: $NEW_RELIC_API_KEY" -d @request.json` (EU accounts: `api.eu.newrelic.com`).

```
# 1. Find an entity's GUID
{ actor { entitySearch(query: "name = 'petclinic' AND domain = 'APM' AND type
= 'APPLICATION'") {
    results { entities { guid name } } } } }

# 2. Record a deployment from a pipeline
mutation { changeTrackingCreateDeployment(deployment: {
    entityGuid: "ENTITY_GUID", version: "4.1.0", commit: "9f3c2ab",
    changelog: "Owner search index and telemetry", user: "ci-pipeline",
    deepLink: "https://ci.example.com/builds/412", deploymentType: BASIC }) {
  deploymentId timestamp } }

# 3. A latency service level: 99 % of owner searches under 0.1 s over 7 days
mutation { serviceLevelCreate(entityGuid: "ENTITY_GUID", indicator: {
    name: "Owner search latency",
    events: { accountId: 1234567,
      validEvents: { from: "Transaction", where: "entityGuid = 'ENTITY_GUID'
AND name LIKE '%OwnerSearch%'" },
      goodEvents:  { from: "Transaction", where: "entityGuid = 'ENTITY_GUID'
AND name LIKE '%OwnerSearch%' AND duration < 0.1" } },
    objectives: [{ target: 99.0, timeWindow: { rolling: { count: 7, unit: DAY
} } }] }) {
  id } }
```

Service level NRQL — Module 15

```
-- SLI attainment (sli.id from the service level's page or NerdGraph)
FROM Metric SELECT clamp_max((count(newrelic.sli.valid) -
count(newrelic.sli.bad)) / count(newrelic.sli.valid) * 100, 100)
AS 'SLI attainment' WHERE sli.id = 'SLI_ID' UNTIL 2 minutes ago TIMESERIES
AUTO
```

Error budget left, in percent of the budget, is `100 - (100 - attainment) / (100 - target) × 100`: with a 99.5 % target and 99.8 % attainment, 40 % of the budget is used and 60 % is left.

Kubernetes integration — Module 14

```
helm repo add newrelic https://helm-charts.newrelic.com && helm repo update
helm upgrade --install newrelic-bundle newrelic/nri-bundle \
  --namespace newrelic --create-namespace \
  --set global.licenseKey="$NEW_RELIC_LICENSE_KEY" \
  --set global.cluster=nr-lab-kind \
  --set newrelic-infrastructure.privileged=true \
  --set kube-state-metrics.enabled=true \
  --set kubeEvents.enabled=true \
  --set global.lowDataMode=true
```

Integration go-live checklist — Module 14

- [ ] Each destination tested from New Relic; the receiving team confirms the message arrived where they expected.
- [ ] Severity-to-tool routing written down and matching the workflows.
- [ ] PagerDuty urgency and ServiceNow field mapping agreed with their owners.
- [ ] API keys and integration credentials in secret stores, owned by a service account, rotated on a schedule.
- [ ] Deployment markers recorded by the pipeline for every service, with the same version the agent reports.
- [ ] Correlation decisions reviewed after the first two weeks; notifications per real problem measured.

Knowledge check, answers and sources

Questions

1. Why does `SELECT count(*) FROM Transaction WHERE error IS true` not give you an error *rate*, and how do you get the rate in one query?
2. You need the average error rate across 40 hosts, where each host counts equally regardless of traffic. Which NRQL feature do you use?
3. Monday's p95 looks 30 % worse than yesterday's. What comparison should you make instead, and why?
4. In a four-step funnel, step 2 has 9,000 sessions and step 3 has 2,000. What does that tell you, and what does it not tell you?
5. Average response time is 280 ms and p95 is 310 ms, but users complain about 4-second pages. What do you check next?
6. Why does the order of conditions matter in `FACET CASES`?

7. What is the difference between a destination and a workflow, and why is one *issue* — not one incident — the unit that gets sent?
8. Propose a routing for critical and warning issues across Slack, PagerDuty and ServiceNow.
9. What should a deployment marker from a CI/CD pipeline contain, and where does the API key live?
10. Name the recommended way to send AWS metrics to New Relic, and what makes infrastructure data more useful than extra charts.
11. Correlation merged two unrelated outages into one issue. What do you do?
12. A journey has an SLO of 99.9 % over 28 days and served 2,000,000 requests. How many bad requests does the error budget allow, and what does 2 % of the budget in one hour mean?
13. The error budget is 80 % spent with ten days of the window left. According to a typical budget policy, what happens next?
14. Write the six parts of an improvement recommendation, and say which part most presentations leave out.

Answers

1. `WHERE` removes the successful transactions before counting, so only the numerator is left. Use `filter()` for the numerator over the full set: `SELECT filter(count(*), WHERE error IS true) / count(*) * 100 FROM Transaction` — or `percentage(count(*), WHERE error IS true)`.
2. A **nested aggregation**: an inner query computes the error rate `FACET host`, and the outer query averages those rows.
3. Compare with **the same weekday last week** (`COMPARE WITH 1 week ago`): traffic and behaviour differ by day of the week, so Monday against Sunday compares two different loads.
4. Most sessions that reached step 2 did not reach step 3 — that is where to look. It does not tell you *why*: open session traces, JS errors and the page's timings for that step.
5. The distribution: a `histogram()` and higher percentiles (p99, max) — and the browser side, because a 4-second *page* can be a fast server response plus slow network, rendering or JavaScript.
6. Each event is counted in the **first** case it matches; wide cases first would swallow the narrow ones.
7. A destination is the connection to a tool; a workflow chooses which issues go to which destinations and what they carry. An issue groups the related incidents of one problem, so one problem produces one notification.

8. Critical: PagerDuty to page, ServiceNow for the record, the team's Slack channel for awareness. Warning: Slack only. Every message carries the runbook and issue links and an NRQL enrichment.
9. Version (matching `service.version`), commit, changelog, user or pipeline, and a deep link to the build. The API key is a secret of the pipeline (for example a GitHub Actions secret), never in the file.
10. **CloudWatch metric streams** through Kinesis Data Firehose. The relationships: entities for hosts, pods, clusters and cloud resources are related to the services running on them, so you can go from a slow service to the resource under it.
11. Split or review the issue, then review the decision that merged them — narrow it or disable it — and check other issues it merged. Correlation needs human review, especially in the first weeks.
12. 0.1 % of 2,000,000 = **2,000** bad requests. 2 % of the budget in one hour is 40 bad requests in an hour — a burn rate of 0.02 × 672 = 13.44, which would spend the whole budget in about 50 hours.
13. Under 25 % left: freeze non-essential releases and put reliability work first, as agreed in advance with the business.
14. Observation, impact, cause, recommendation, effort and risk, verification. **Verification** — how we will know it worked — is the part most often left out.

Sources

- New Relic docs — NRQL syntax, clauses and functions
- New Relic docs — Service level management: create, consume and alerting on service levels
- New Relic docs — NerdGraph service level examples
- New Relic docs — Change tracking with NerdGraph
- New Relic docs — Kubernetes integration (FedRAMP Helm example)
- NRQL Reference Guide from this course: NRQL Reference Guide — Commands, Functions and Extraction Queries

UI labels, integration set-up screens and API fields change over time; check the current New Relic documentation for each integration before go-live.